

Smart Notification Filtering AI Agent

Reducing Digital Overload Using LLM-Driven Decisions

Name	Sai Gayatri Sunil
College	Chaitanya Bharathi Institute of Technology
Branch	Computer Science and Engineering
Year of Study	2 nd year
Phone No.	7337506066
Email	sai15gayatri@gmail.com

INDEX

Click on each title to navigate to the starting slide of the topic

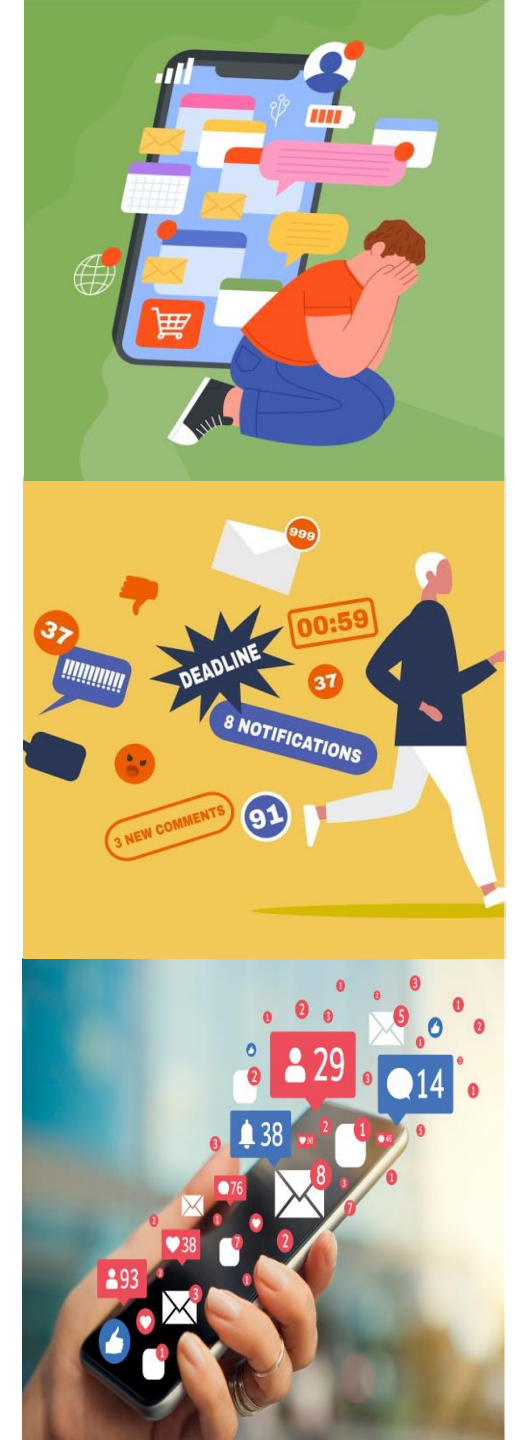
S.No.	Title	Starting Slide	Ending Slide	No. of Slides
1	The Problem	3	3	1
2	Why Traditional Filters Fail	4	4	1
3	Proposed Solution	5	5	1
4	Working of the AI Agent	6	10	5
4	Tools & Technologies Used	11	11	1
5	Code Explanation	12	19	8
6	Results	20	25	6
7	Business Use Cases	26	26	1
8	Learning Outcomes	27	27	1
9	Future Enhancements	27	27	1
10	Colab Notebook Link	27	27	1

THE PROBLEM

Information Overload in the Digital Age

- Users receive **hundreds of notifications daily** across apps and platforms
- Most notifications are **low-priority or irrelevant**
- Important information often gets **missed due to noise**
- Constant interruptions increase **stress, anxiety, and cognitive fatigue**, reducing the ability to focus deeply.
- Repeatedly judging notification importance drains mental energy, leading to **poor prioritization and burnout**.

Therefore, the problem is not notifications themselves, but the lack of intelligent, adaptive decision-making behind them.

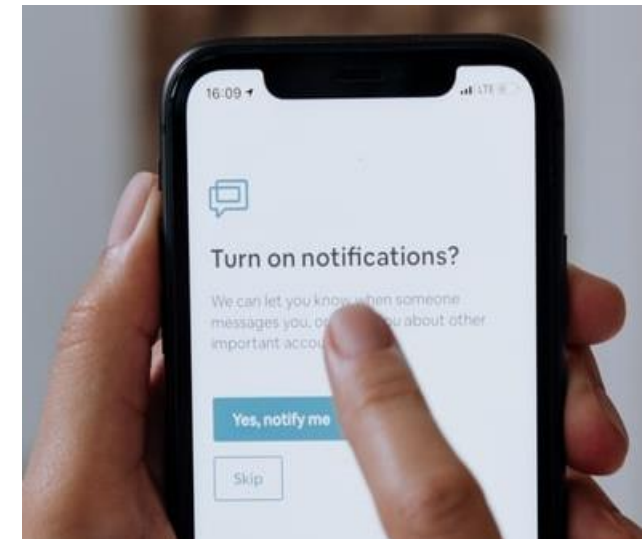
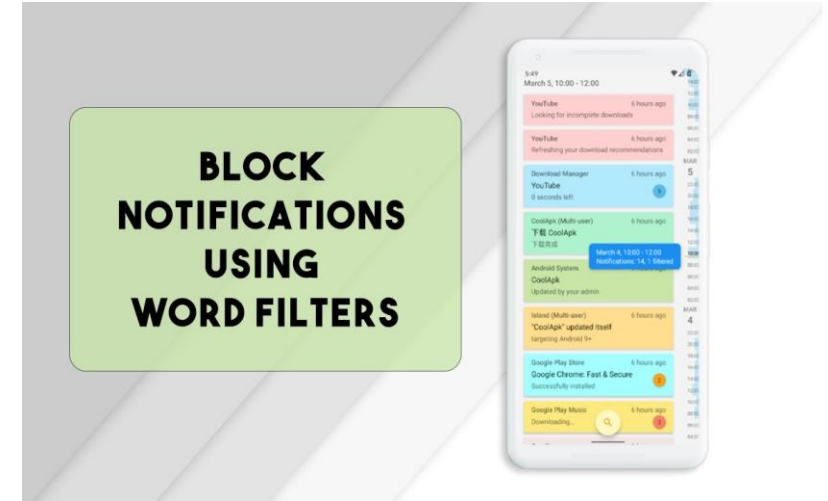


WHY TRADITIONAL FILTERS FAIL

Rules Can't Understand Context

- Same notification can be:
 - Urgent for one user
 - Irrelevant for another
- Keyword-based systems:
 - Miss semantic meaning
 - Cannot adapt to user behavior

Thus, we need a system that understands meaning and adapts over time



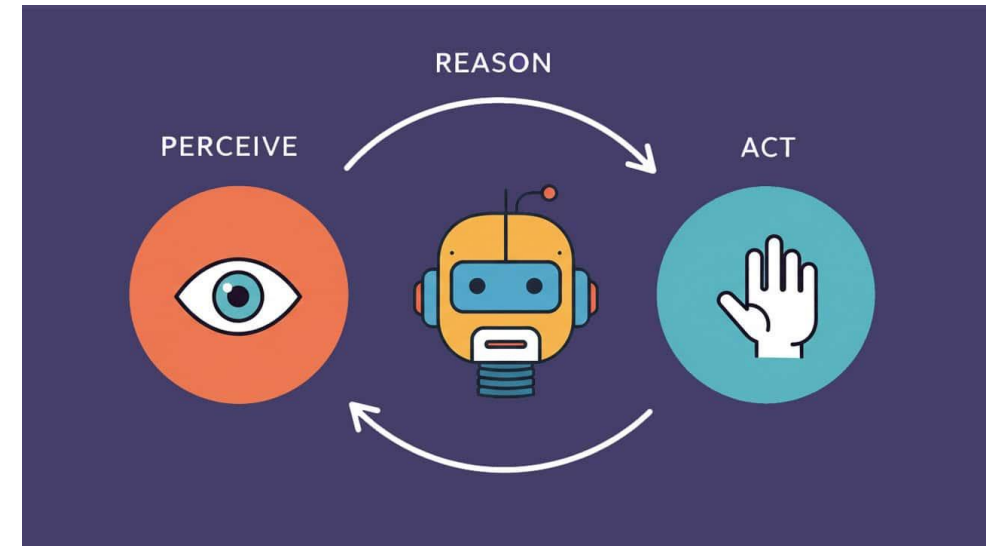
PROPOSED SOLUTION

An AI Agent That Thinks Before Notifying

An AI Agent that:

- ✓ Observes incoming notifications
- ✓ Makes semantic decisions using an LLM
- ✓ Acts by prioritizing or deferring notifications
- ✓ Learns continuously through user feedback

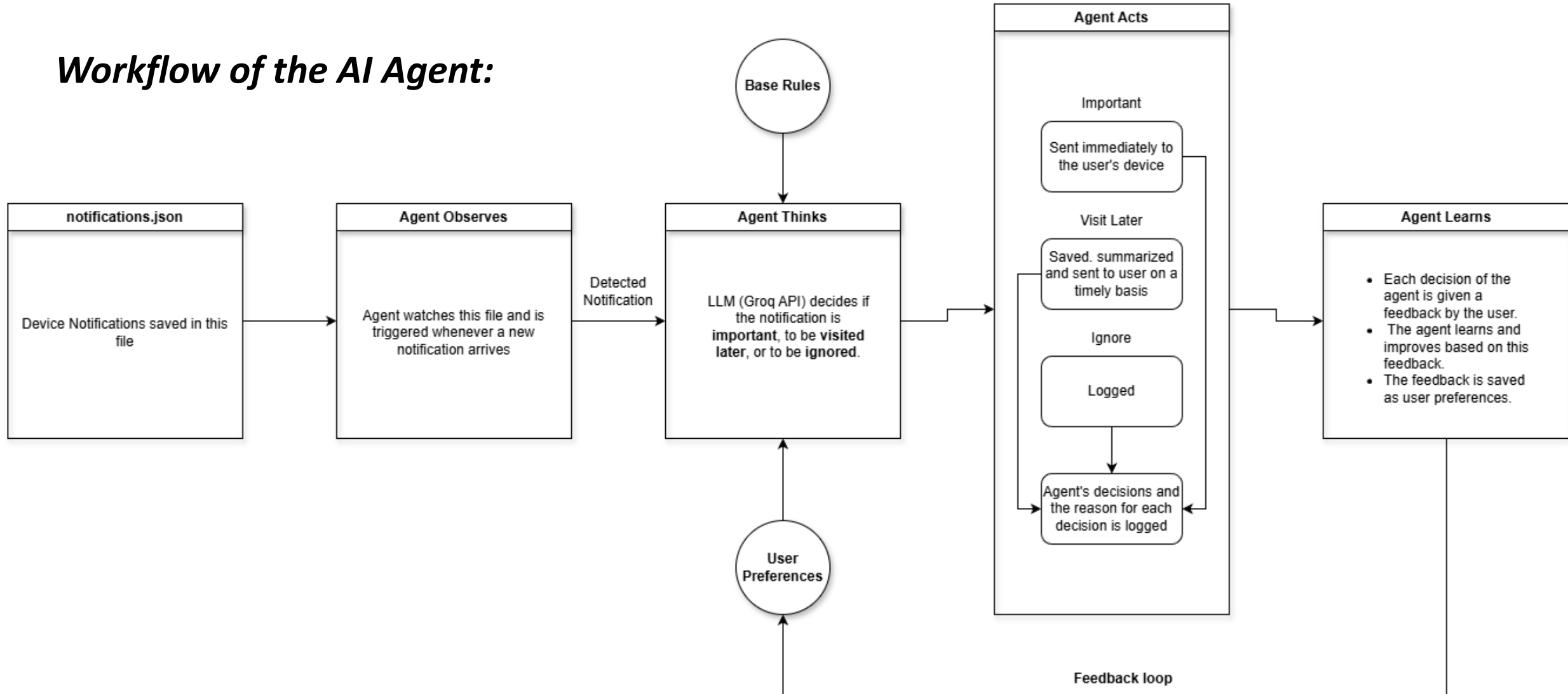
Here, the ability of LLM to understand natural language and context is utilized to make intelligent decisions



Agent loop: Observe -> Think -> Act

WORKING OF THE AI AGENT

Workflow of the AI Agent:



NOTIFICATIONS

Notifications in JSON format



- The notifications in this project are **conceptually simulated** by using a file (notifications.json) and all the notifications are in JSON format.
- **notifications.json** represents the **stream of notifications arriving into the device** (phone/tablet/laptop/PC) from various sources (Gmail, LinkedIn, Shopping apps, Bank apps, Social Media apps etc.).
- This file/stream is **watched by the AI agent** before it appears in the device as a notification.
- Real world notifications are essentially **JSON (JavaScript Object Notation) objects**.
- **JSON** is a lightweight, text-based **data-interchange format** that is easy for humans to read and write, and simple for machines (computers and software) to parse and generate. It is widely used in modern web development for transmitting data between a server and a web application.
- Therefore, in this project, all the notifications are **JSON objects**.
- The **json** library of Python is used to work with JSON data format.
- Each new notification acts as a **trigger** to the AI agent.

DECISION MAKING WITH LLM

LLM-Based Semantic Reasoning



The LLM (LLaMa 3) decides by the semantic understanding and reasoning of:

- **Base Rules:** The basic, generic rules and policies to be followed by the LLM while making the decision. Similar to real world company or model generic rules and policies.
- **User Preferences:** The preferences learnt by the agent over the time by user feedback. It makes sure that the decisions are user-specific.
- **Notification content:** The agent understands the context of the notification that is in natural, human language, thus figuring out the importance and relevance of the notification.

LLM decides the category of each notification. The categories are:

1. **IMPORTANT:** These notifications require immediate attention of the user or are highly correlated to their preferences.
2. **VISIT LATER:** These are the notifications that do not require the urgent attention of the user, but the user might want to check them out later, in their leisure time.
3. **IGNORE:** These notifications are to be ignored (spam, promotions etc.).

ACTIONS

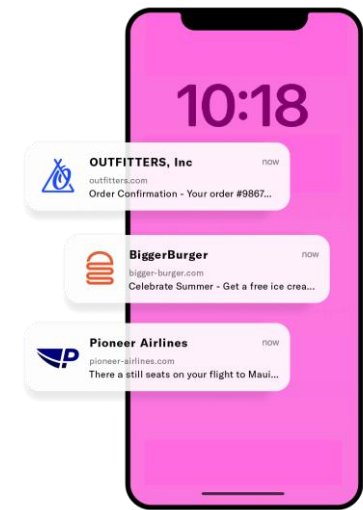
Actions that respect the user's attention and interests

Based on the LLM's decisions, the notifications that are in the category of:

1. **Important** are immediately sent to the device (conceptually simulated by **alerts.json**)
2. **Visit Later** are saved (in **digest.json**) and a summary of the detected Visit Later notifications is sent to the device (**alerts.json**) as a notification, so that, the user can decide on the notifications later.
3. **Ignore** are never sent to the user.

All the notifications sent to alerts.json are the only notifications the user would view as device notifications. All the other notifications are hidden, but can be viewed later by the user in the app or it's logs.

All the decisions along with the reason why LLM made that decision is stored in app logs, so that the AI system is transparent and explainable.



FEEDBACK

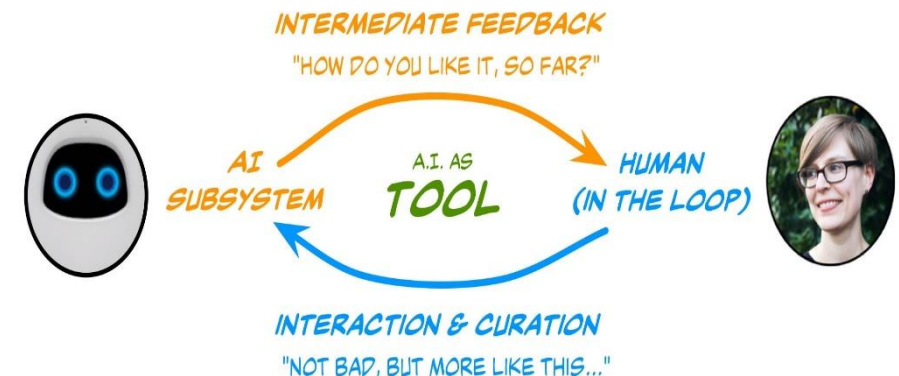
The agent improves without retraining the model



- The model uses **In-Context Learning (ICL)** and **Human-In-The-Loop (HITL)** to constantly improve and decide closer to the user's preferences.
- **In-context learning (ICL)** is a powerful capability of large language models (LLMs) where they learn to perform new tasks from **examples provided directly in the prompt**, without any changes to the model's underlying parameters or weights.
- ICL is achieved by sending the user preferences (stored in **user_preferences.json**) in the LLM prompt.
- A **Human-in-the-Loop (HITL)** approach integrates human intelligence and **feedback** into AI/ML workflows, creating a collaborative system where humans train, validate, fine-tune, and **correct AI models**.
- HITL ensures accuracy, fairness, and **better real-world performance** by combining AI's speed with human judgment and context.
- HITL approach is adopted by **asking user feedback** for every decision made by the AI agent.
- This feedback is then stored in **user_preferences.json** and is used by the LLM to make future decisions close to user's preferences.
- This approach helps combat **AI hallucination** and aims to provide **user satisfaction and accuracy**.

TOOLS & TECHNOLOGIES USED

- ❑ **Programming:** Python
- ❑ **AI Model:** LLaMA 3 (via Groq API)
- ❑ **Platform:** Google Colab
- ❑ **Techniques:**
 - Prompt Engineering
 - JSON based memory and state
 - Human In The Loop (HITL) learning
 - In-Context Learning (ICL)



CODE EXPLANATION

Creating the necessary files

```
%%writefile notifications.json  
[]
```

To receive incoming notifications

```
%%writefile agent_state.json  
{  
  "last_processed_id": -1  
}
```

To make sure old notifications are not re-processed

```
%%writefile logs.json  
[]  
To log all agent decisions
```

```
%%writefile user_preferences.json  
{"rules": []}  
To store user preferences through feedback
```

```
%%writefile base_rules.json  
{  
  "principles": [  
    "Time-sensitive academic, career, or safety-related notifications are usually important",  
    "Promotional or marketing notifications are usually low priority",  
    "Deadlines, exams, applications, meetings, and urgent updates deserve higher attention",  
    "User attention is valuable and should not be interrupted unnecessarily",  
    "When unsure, prefer 'Visit Later' instead of 'Ignore'"  
  ]  
}
```

To store base rules (especially important to guide the agent when user preferences are empty)

```
%%writefile alerts.json  
[]  
To store important notifications
```

```
%%writefile digest.json  
[]  
To store notifications to be visited later
```

```
%%writefile digest_state.json  
{  
  "last_summary_time": 0  
}  
To aid the timely notification on Visit Later notifications
```

CODE EXPLANATION

Sending notifications

The list of notifications used in this conceptual simulation (**INPUT**):

```
ALL_NOTIFICATIONS = [  
    {"id": 1, "source": "Instagram", "content": "Someone liked your recent post."},  
    {"id": 2, "source": "LinkedIn", "content": "New post: Looking for interns with Python and ML skills."},  
    {"id": 3, "source": "College Portal", "content": "Reminder: Data Structures assignment submission closes tonight."},  
    {"id": 4, "source": "LinkedIn", "content": "New post: Top 10 skills every developer must learn in 2025"},  
    {"id": 5, "source": "HDFC Bank", "content": "Rs. 50,000 credited from your account. Wasn't you? Check out ways to secure your account."},  
    {"id": 6, "source": "Myntra", "content": "New Year Sale is LIVE! Get 50% off on all products. Offer live only today. Grab now!"},  
    {"id": 7, "source": "Gmail", "content": "Your monthly electricity bill is ready to view."},  
    {"id": 8, "source": "LinkedIn", "content": "New post: CISCO is hiring!"},  
    {"id": 9, "source": "Amazon", "content": "Order shipped. Track your order in the app."},  
    {"id": 10, "source": "BigBasket", "content": "Your ONE TIME PASSWORD is *****"},  
    {"id": 11, "source": "YouTube", "content": "New video from MyTravelVlogs."},  
    {"id": 12, "source": "LinkedIn", "content": "New post: Smart India Hackathon is live!"},  
    {"id": 13, "source": "Gmail", "content": "Water bill due in 3 days."},  
    {"id": 14, "source": "LinkedIn", "content": "New post: Looking for students proficient in Python."},  
    {"id": 15, "source": "Nykaa", "content": "Urgent: Flat 50% off on all products. Hurry up! Time is ticking!"}  
]
```

Here, we are conceptually simulating the arrival of notifications into a device. These notifications, in JSON format (to simulate real world data transfer), would be read by the agent.

Code to send and save notifications in **notifications.json**:

```
import json  
import time  
import os  
  
NOTIFICATIONS_FILE = "notifications.json"  
  
if not os.path.exists(NOTIFICATIONS_FILE):  
    with open(NOTIFICATIONS_FILE, "w") as f:  
        json.dump([], f)  
  
for notif in ALL_NOTIFICATIONS:  
    with open(NOTIFICATIONS_FILE, "r") as f:  
        existing = json.load(f)  
  
    existing.append(notif)  
  
    with open(NOTIFICATIONS_FILE, "w") as f:  
        json.dump(existing, f, indent=2)  
  
    print(f"New notification emitted: {notif['content']}")  
    time.sleep(1)
```

CODE EXPLANATION

Initializing the LLM (the brain of the agent)

```
!pip install -q groq
```

Installing Groq Python Library

```
from google.colab import userdata
import os
os.environ["GROQ_API_KEY"] = userdata.get('groqapikey')
```

Configuring the API key as an environment variable in a secure way

```
import json
import time
import os
import re
from groq import Groq
```

```
# ----- Groq Client -----
```

```
client = Groq(
    api_key=os.environ.get("GROQ_API_KEY"),
)
```

Initializing Groq Client

CODE EXPLANATION – Functions used by Agent

```
# ----- Utilities -----  
  
def load_json(path):  
    with open(path, "r") as f:  
        return json.load(f)  
  
def save_json(path, data):  
    with open(path, "w") as f:  
        json.dump(data, f, indent=2)  
  
def extract_json(text):  
    """  
    Safely extract JSON object from LLM response.  
    """  
    match = re.search(r"\{.*\}", text, re.DOTALL)  
    if not match:  
        raise ValueError("No JSON found in LLM response")  
    return json.loads(match.group())
```

Utility functions to be used throughout the program. `load_json()` reads JSON file, `save_json()` writes data into a JSON file, and `extract_json()` safely extracts JSON object from LLM response (as we can't guarantee the LLM to respond only in JSON. Any other format could result in `JSONDecodeError` as all the functions in the program are written for JSON format only)

```
# ----- Detection -----  
  
def detect_new_notifications():  
    notifications = load_json("notifications.json")  
    agent_state = load_json("agent_state.json")  
  
    last_id = agent_state["last_processed_id"]  
    new = [n for n in notifications if n["id"] > last_id]  
  
    return new, agent_state  
  
def update_agent_state(agent_state, new_id):  
    agent_state["last_processed_id"] = new_id  
    save_json("agent_state.json", agent_state)
```

The above functions help the agent to detect new notifications. The notifications are tracked using an id unique to every notification. The agent state tracks the last processed notification. This prevents the agent from reprocessing the old notifications.

```
# ----- LLM Decision -----

SYSTEM_PROMPT = """
You are an AI notification-filtering agent.

You MUST respond ONLY in valid JSON.
Do NOT include explanations.
Do NOT include markdown.
Do NOT include extra text.

Required schema:
{
  "category": "Important | Visit Later | Ignore",
  "reason": "string",
  "confidence": number between 0 and 1
}
"""
```

System prompt to ensure response in correct format

```
def llm_decide(notification, base_rules, user_prefs):
    """
    LLM-based semantic decision making.
    Robust against malformed responses.
    """
```

```
    user_prompt = f"""
    BASE PRINCIPLES:
    {base_rules['principles']}
```

```
    USER PREFERENCES (HIGH PRIORITY - OVERRIDE BASE RULES):
    {user_prefs['rules']}
```

```
    NOTIFICATION:
    "{notification}"
    """
```

```
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_prompt}
        ],
        temperature=0.2
    )
```

```
    raw = response.choices[0].message.content
```

User prompt to feed in base rules and user preferences to guide the LLM's decision. The LLM used in this agent is llama-3.3-70b-versatile.

```
try:
    return extract_json(raw)
except Exception:
    # Fallback so agent NEVER crashes
    return {
        "category": "Visit Later",
        "reason": "LLM response could not be parsed safely",
        "confidence": 0.5
    }
```

If LLM responds in any other format other than JSON

----- Action Layer -----

```
def act_on_decision(notification, decision):  
    if decision["category"] == "Important":  
        alerts = load_json("alerts.json")  
        alerts.append(notification)  
        save_json("alerts.json", alerts)  
  
    elif decision["category"] == "Visit Later":  
        digest = load_json("digest.json")  
        digest.append(notification)  
        save_json("digest.json", digest)
```



This function acts based upon the LLM's decision.

- ✓ ***If LLM decides that the notification is Important, it is immediately sent to the device (conceptually simulated by appending alerts.json)***
- ✓ ***If LLM decides that the notification can be visited later, then, it is saved for later review (conceptually simulated by appending digest.json)***
- ✓ ***If LLM decides to ignore the notification, then no action is taken other than logging the decision (to be done in Agent Loop)***

----- Feedback (HITL) -----

```
def apply_feedback(notification_text, decision, user_prefs):  
    print(f"\nNotification: {notification_text}")  
    print(f"Agent decision: {decision['category']}")  
    feedback = input("Was this correct? (c = correct / w = wrong): ").lower()  
  
    if feedback == "w":  
        correct = input("Correct category (Important / Visit Later / Ignore): ").strip()  
        rule = f"When a notification is like '{notification_text}', it should be '{correct}'."  
  
        user_prefs["rules"].append(rule)  
  
        print("Feedback stored.")  
  
    return user_prefs
```



This function asks for user feedback after every decision. If the decision is wrong, it asks for the right decision. It then saves these right decisions as user preferences and applies them for future notifications. Thus, gradually, the agent learns about the user and decides accordingly

```
# ----- Summarising Visit Later -----

def llm_summarize_digest(digest_items, base_rules, user_prefs):
    if not digest_items:
        return None

    joined = "\n".join([f"- {n['content']}" for n in digest_items])

    prompt = f"""
You are an AI assistant helping reduce notification overload.

USER PREFERENCES:
{user_prefs['rules']}

TASK:
Summarize the following notifications into a concise, helpful message.
Group similar items.
Focus on what the user should know or act on.

NOTIFICATIONS:
{joined}

Respond ONLY with plain text.

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.3
)

return response.choices[0].message.content.strip()
```

Uses LLM to summarize all the Visit Later notifications stored in digest.json

```
# ----- Timed Summary -----

SUMMARY_INTERVAL = 10 # seconds (short for demo)

def check_and_send_digest(base_rules, user_prefs):
    digest = load_json("digest.json")
    digest_state = load_json("digest_state.json")

    now = time.time()

    if now - digest_state["last_summary_time"] < SUMMARY_INTERVAL:
        return

    if not digest:
        return

    summary = llm_summarize_digest(digest, base_rules, user_prefs)
    if not summary:
        return

    alerts = load_json("alerts.json")
    alerts.append({
        "type": "VisitLaterSummary",
        "content": summary,
        "timestamp": now
    })

    save_json("alerts.json", alerts)

    # clear digest after summary
    save_json("digest.json", [])

    digest_state["last_summary_time"] = now
    save_json("digest_state.json", digest_state)
```

Sends the summarized notification to the device (alerts.json) every 10 seconds (for demo purpose). digest_state is used to keep track of the time the notification was last sent. At the end, the digest.json is cleared to avoid repeated sending of the same summarized notification

```
# ----- Agent Loop -----
```

```
while True:
```

```
    new_notifications, agent_state = detect_new_notifications()
```

```
    if new_notifications:
```

```
        base_rules = load_json("base_rules.json")
```

```
        user_prefs = load_json("user_preferences.json")
```

```
        logs = load_json("logs.json")
```

Loads required data and files to be used by the agent

```
    print(f"\nAgent detected {len(new_notifications)} new notification(s)")
```

————— ***Prints number of detected notifications***

```
    for notif in new_notifications:
```

```
        decision = llm_decide(notif["content"], base_rules, user_prefs)
```

```
        user_prefs = apply_feedback(notif["content"], decision, user_prefs)
```

```
        act_on_decision(notif, decision)
```

Decides, acts on that decision, and receives user feedback

```
        log_entry = {
```

```
            "notification_id": notif["id"],
```

```
            "content": notif["content"],
```

```
            "decision": decision["category"],
```

```
            "reason": decision["reason"],
```

```
            "confidence": decision["confidence"]
```

```
        }
```

```
        logs.append(log_entry)
```

```
        save_json("logs.json", logs)
```

```
        save_json("user_preferences.json", user_prefs)
```

```
        update_agent_state(agent_state, notif["id"])
```

Logs the LLM decision, LLM's confidence in that decision, and it's reason for that decision in logs.json

```
    else:
```

```
        print("No new notifications")
```

————— ***Prints when no new notifications are detected***

```
    check_and_send_digest(base_rules, user_prefs)
```

————— ***Sends timely Visit Later summary notifications***

```
    time.sleep(5)
```

————— ***Sleeps for 5 seconds to simulate real world working***

RESULTS

Input – notifications.json

notifications.json represents the stream of incoming notifications into the device.

notifications.json

```
1 [
2   {
3     "id": 1,
4     "source": "Instagram",
5     "content": "Someone liked your recent post."
6   },
7   {
8     "id": 2,
9     "source": "LinkedIn",
10    "content": "New post: Looking for interns with Python and ML skills."
11  },
12  {
13    "id": 3,
14    "source": "College Portal",
15    "content": "Reminder: Data Structures assignment submission closes tonight."
16  },
17  {
18    "id": 4,
19    "source": "LinkedIn",
20    "content": "New post: Top 10 skills every developer must learn in 2025"
21  },
22  {
```

```
--
63  {
64    "id": 13,
65    "source": "Gmail",
66    "content": "Water bill due in 3 days."
67  },
68  {
69    "id": 14,
70    "source": "LinkedIn",
71    "content": "New post: Looking for students proficient in Python."
72  },
73  {
74    "id": 15,
75    "source": "Nykaa",
76    "content": "Urgent: Flat 50% off on all products. Hurry up! Time is ticking!"
77  }
78 ]
```

notifications.json

```
22 {
23   "id": 5,
24   "source": "HDFC Bank",
25   "content": "Rs. 50,000 credited from your account. Wasn't you? Check out ways to secure your account."
26 },
27 {
28   "id": 6,
29   "source": "Myntra",
30   "content": "New Year Sale is LIVE! Get 50% off on all products. Offer live only today. Grab now!"
31 },
32 {
33   "id": 7,
34   "source": "Gmail",
35   "content": "Your monthly electricity bill is ready to view."
36 },
37 {
38   "id": 8,
39   "source": "LinkedIn",
40   "content": "New post: CISCO is hiring!"
41 },
```

notifications.json

```
40   "content": "New post: CISCO is hiring."
41 },
42 {
43   "id": 9,
44   "source": "Amazon",
45   "content": "Order shipped. Track your order in the app."
46 },
47 {
48   "id": 10,
49   "source": "BigBasket",
50   "content": "Your ONE TIME PASSWORD is *****."
51 },
52 {
53   "id": 11,
54   "source": "YouTube",
55   "content": "New video from MyTravelVlogs."
56 },
57 {
58   "id": 12,
59   "source": "LinkedIn",
60   "content": "New post: Smart India Hackathon is live!"
61 },
62 }
```

RESULTS

Output – alerts.json

alerts.json represents the notifications sent to the device by the AI agent after filtering.

These are the notifications that will actually be viewed as device notifications by the user.

alerts.json

```
1 [
2   {
3     "id": 3,
4     "source": "College Portal",
5     "content": "Reminder: Data Structures assignment submission closes tonight."
6   },
7   {
8     "id": 5,
9     "source": "HDFC Bank",
10    "content": "Rs. 50,000 credited from your account. Wasn't you? Check out ways to secure your account."
11  },
12  {
13    "id": 8,
14    "source": "LinkedIn",
15    "content": "New post: CISCO is hiring!"
16  },
17  {
18    "id": 10,
19    "source": "BigBasket",
20    "content": "Your ONE TIME PASSWORD is *****."
21  },
22  {
```

Important notifications

alerts.json

```
21 },
22 {
23   "id": 12,
24   "source": "LinkedIn",
25   "content": "New post: Smart India Hackathon is live!"
26 },
27 {
28   "id": 13,
29   "source": "Gmail",
30   "content": "Water bill due in 3 days."
31 },
32 {
33   "id": 14,
34   "source": "LinkedIn",
35   "content": "New post: Looking for students proficient in Python."
36 },
37 {
38   "type": "VisitLaterSummary",
39   "content": "You have 3 new notifications: \n1 important job posting for interns with Python and ML skills, \n1
40   "timestamp": 1767090983.6365592
41 }
42 ]
```

Visit Later Summary Notification

RESULTS

Output – logs.json

*logs.json is opened in VS
Code here for clearer view*

Each log consists of:

- 1. Notification ID***
- 2. Notification content***
- 3. LLM decision***
- 4. LLM's reason for making that decision***
- 5. LLM's confidence on that decision***

```
1  [
2  {
3    "notification_id": 1,
4    "content": "Someone liked your recent post.",
5    "decision": "Ignore",
6    "reason": "The notification is about a social media like, which is not time-sensitive or related to academic, career, or safety",
7    "confidence": 0.9
8  },
9  {
10   "notification_id": 2,
11   "content": "New post: Looking for interns with Python and ML skills.",
12   "decision": "Visit Later",
13   "reason": "The notification is related to a career opportunity but does not contain time-sensitive or urgent information",
14   "confidence": 0.7
15 },
16 {
17   "notification_id": 3,
18   "content": "Reminder: Data Structures assignment submission closes tonight.",
19   "decision": "Important",
20   "reason": "Time-sensitive academic deadline",
21   "confidence": 0.9
22 },
23 {
24   "notification_id": 4,
25   "content": "New post: Top 10 skills every developer must learn in 2025",
26   "decision": "Visit Later",
27   "reason": "The notification is about a general topic and does not contain urgent or time-sensitive information, but may be relevant to the user's interests",
28   "confidence": 0.7
29 },
30 ],
31 {
32   "notification_id": 5,
33   "content": "Rs. 50,000 credited from your account. Wasn't you? Check out ways to secure your account.",
34   "decision": "Important",
35   "reason": "The notification mentions a significant financial transaction and implies potential security concerns, which requires immediate attention.",
36   "confidence": 0.9
37 },
38 {
39   "notification_id": 6,
40   "content": "New Year Sale is LIVE! Get 50% off on all products. Offer live only today. Grab now!",
41   "decision": "Ignore",
42   "reason": "Promotional notification with limited-time offer",
43   "confidence": 0.9
44 },
45 {
46   "notification_id": 7,
47   "content": "Your monthly electricity bill is ready to view.",
48   "decision": "Visit Later",
49   "reason": "The notification is not time-sensitive or related to academic, career, or safety, but is not promotional or marketing",
50   "confidence": 0.7
51 },
52 {
53   "notification_id": 8,
54   "content": "New post: CISCO is hiring!",
55   "decision": "Important",
56   "reason": "Notification matches high-priority user preference for job postings",
57   "confidence": 0.9
58 },
59 ]
```

RESULTS

Output – logs.json

*logs.json is opened in VS Code
here for clearer view*

Each log consists of:

- 1. Notification ID**
- 2. Notification content**
- 3. LLM decision**
- 4. LLM's reason for making that decision**
- 5. LLM's confidence on that decision**

```
57 },
58 {
59   "notification_id": 9,
60   "content": "Order shipped. Track your order in the app.",
61   "decision": "Ignore",
62   "reason": "The notification is a standard shipping update and does not contain time-sensitive or urgent information.",
63   "confidence": 0.8
64 },
65 {
66   "notification_id": 10,
67   "content": "Your ONE TIME PASSWORD is *****.",
68   "decision": "Important",
69   "reason": "Time-sensitive safety-related notification",
70   "confidence": 0.9
71 },
72 {
73   "notification_id": 11,
74   "content": "New video from MyTravelVlogs.",
75   "decision": "Ignore",
76   "reason": "Notification is about a new video from a vlog channel, which is likely promotional or entertainment-related",
77   "confidence": 0.8
78 },
79 {
80   "notification_id": 12,
81   "content": "New post: Smart India Hackathon is live!",
82   "decision": "Important",
83   "reason": "The notification is about a time-sensitive event, Smart India Hackathon, which is likely related to academic or career opportunities",
84   "confidence": 0.8
85 },
86 {
87   "notification_id": 13,
88   "content": "Water bill due in 3 days.",
89   "decision": "Important",
90   "reason": "Time-sensitive notification regarding a bill due in 3 days",
91   "confidence": 0.9
92 },
93 {
94   "notification_id": 14,
95   "content": "New post: Looking for students proficient in Python.",
96   "decision": "Important",
97   "reason": "Notification matches high-priority user preference for Python skills",
98   "confidence": 0.9
99 },
100 {
101   "notification_id": 15,
102   "content": "Urgent: Flat 50% off on all products. Hurry up! Time is ticking!",
103   "decision": "Ignore",
104   "reason": "Promotional notification with no relevance to academic, career, or safety-related matters",
105   "confidence": 0.9
106 }
107 ]
```

RESULTS

Feedback

...

Agent detected 15 new notification(s)

Notification: Someone liked your recent post.

Agent decision: Ignore

Was this correct? (c = correct / w = wrong): c

Notification: New post: Looking for interns with Python and ML skills.

Agent decision: Visit Later

Was this correct? (c = correct / w = wrong): w

Correct category (Important / Visit Later / Ignore): Important

Feedback stored.

Notification: Reminder: Data Structures assignment submission closes tonight.

Agent decision: Important

Was this correct? (c = correct / w = wrong): c

Notification: New post: Top 10 skills every developer must learn in 2025

Agent decision: Visit Later

Was this correct? (c = correct / w = wrong): c

Notification: Rs. 50,000 credited from your account. Wasn't you? Check out ways to secure your account.

Agent decision: Important

Was this correct? (c = correct / w = wrong): c

⋮

Upto 15 notifications

RESULTS

HITL and ICL demonstration:

For an earlier notification (ID: 2), the LLM gave the **decision** as shown aside. It categorized the notification involving a hiring post requiring Python and ML skills as **Visit Later**, as it considered the notification to not be an urgent information but something that the user might be interested in to view later.

```
{  
  "notification_id": 2,  
  "content": "New post: Looking for interns with Python and ML skills.",  
  "decision": "Visit Later",  
  "reason": "The notification is related to a career opportunity but  
does not contain time-sensitive or urgent information",  
  "confidence": 0.7  
},
```

Then, the **user corrected** the LLM as shown aside. The user wanted the notification **to be categorized as Important** and not as Visit Later. The feedback was then noted by the agent in user preferences.

```
Notification: New post: Looking for interns with Python and ML skills.  
Agent decision: Visit Later  
Was this correct? (c = correct / w = wrong): w  
Correct category (Important / Visit Later / Ignore): Important  
Feedback stored.
```

Later, for a similar notification (ID: 14), the LLM gave the **decision** as **Important**, on the basis of **user preferences**. The reason for the decision reflects that the decision was made based on the user's high-priority preference for Python skills related notifications.

```
{  
  "notification_id": 14,  
  "content": "New post: Looking for students proficient in Python.",  
  "decision": "Important",  
  "reason": "Notification matches high-priority user preference for  
Python skills",  
  "confidence": 0.9  
},
```

BUSINESS USE CASES

Work & Office Tools

- Prioritizes important work messages
- Reduces distraction and overload
- Helps employees focus better

Education Platforms

- Highlights exams, deadlines, and class updates
- Pushes non-urgent messages to later
- Prevents missed academic tasks

Finance & Banking Apps

- Flags payments, fraud, and due dates
- Filters marketing notifications
- Improves user trust

Healthcare & Well-Being

- Prioritizes medical and emergency alerts
- Reduces alert fatigue
- Supports mental well-being

Consumer Apps & Devices

- Smart notification filtering at OS/app level
- Sends summaries instead of constant alerts
- Improves user experience

LEARNING OUTCOMES

- ✓ Understanding the concept of AI agents
- ✓ Practical use of LLMs for decision-making
- ✓ Importance of explainability in AI systems
- ✓ Designing adaptive systems using feedback loops
- ✓ Building real-world AI agents in Python

COLAB NOTEBOOK LINK:

<https://colab.research.google.com/drive/1loZjIG80oYTSa92sIKEbkjh-jfsXMFZg?usp=sharing>

FUTURE ENHANCEMENTS

- ✓ Integration with real device notifications
- ✓ Personalized long-term memory per user
- ✓ UI dashboard using Streamlit
- ✓ Multi-user support
- ✓ Enterprise use cases (extending to email, support tickets, alerts, customer queries)
- ✓ Triggering the agent based on notification traffic (low traffic – each notification, higher traffic – a batch of notifications)